Dynamic Programming

Readings

- http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf
- http://math.pnw.edu/~rlkraft/cs33200-2011/chap15/DynamicProgrammingExamples.pdf

Longest Increasing subsequence

Longest Increasing subsequence

- **Input:**
  - A sequence of numbers $a_1, \ldots, a_n$
- **Output:**
  - The **increasing subsequence** of maximum length
    - A **subsequence** is any subset of these numbers taken in the same order
    - **Increasing subsequence**: a subsequence in which the numbers are getting strictly larger
- **Example:**
  - The longest increasing subsequence of **5, 2, 8, 6, 3, 6, 9, 7** is **2, 3, 6, 9**
- **Brute-force solution**

Subproblems and Recursive Formula

- What are the subproblems?
  - Suppose we define the subproblems to be
    - L[i] = Length of longest increasing subsequence of A[1 … i]
    - Example: 3 4 8 5 6
    - Issue: This does not seem to give enough info to combine subproblems
    - We need to be able to get L[n] from L[i]
    - This is not a good definition for a subproblem.

Subproblems and Recursive Formula

- **L[i] = Length of longest increasing subsequence that ends in A[i]**
- Recurrence relation:
  - Given, we end at i, we try all possible numbers A[j] where j <i, and form an increasing subsequence ending at i by concatenating with a longest increasing subsequence ending at j
  - **L[i] = 1 + max { L(j): 0 < j < i and A[j] < A[i] }**

- Final answer:
  - Longest Increasing Subsequence: L(n): **Max L[i]**

$1 \leq i \leq n$

Bottom-Up implementation

Runtime: $O(n^2)$

Iterative implementation:

```
#initialization
for i from 1 to n
    L[i] = 1
for i from 2 to n do
    L[i] = 1
    for j from 1 to i-1 do
        if A[i] > A[j] and L(i) > L(j)+1
            L[i] = L[j]+1
return max (L)
```

Question/Challenge

Implement the memoized version.

Example

For the example given the L[i] are

A: 5, 2, 8, 6, 3, 6, 9, 7

Maximum Contiguous Subarray

Maximum Contiguous Subarray (MCS)

- Input:
  - An array of n integers: A[1 … n]
- Output:
  - Indices i and j that maximizes

$$\sum_{k=i}^{j} A[k]$$

Example: 1 2 3 −7 3 1 1 −4 6 −3 1

Defining subproblems

M(i) = maximum contiguous subsequence sum between A[1] and A[i]

Is this a good one?

Example: 1 2 3 −7 3 1 1 −4 6 −3 1

What is M(7)?

What is M(6)?

Defining subproblems

ME(i) = maximum contiguous subsequence sum of subsequences that end at A[i]

$ME(i) = \max\{ A[i] + ME(i - 1), A[i] \}$ with $ME(1) = A[i]$

$M(n) = \max_{1 \le i \le n} \{ ME(i) \}$

We can also evaluate M in the following way:

$M(i) = \max\{ M(i - 1), ME(i) \} = \max\{ M(i - 1), ME(i - 1) + ai, ai \}$

Maximum Contiguous Subarray

pseudocode.

```
MS(A, n)
          M[1] = max{0, A[1]}
          ME[1] = A[1]
          for i=1 to n
                    ME[i] = max ( A[i], A[i] + ME[i-1] )
                    M[i] = max( ME[i], M[i-1] )

          return max(M[n], ME[n])
```

Runtime: O(n)

Question

For the following array:

1, -7, 4, 0, 2, -1, 3, 0

What would be the value of ME[8] after applying the previous algorithm (dynamic programming)?

A. 8
B. 7
C. 9
D. 0

Question: Maximum independent subsequence

Suppose we are given an array, A, of n positive numbers A = [a1, a2, . . . , an].

An independent subsequence of A is a selection of elements from A such that no two elements in the selection are adjacent in the array. An independent subsequence is kind of the opposite of a contiguous subsequence in the sense that an independent subsequence must be as non-contiguous as possible; no two elements of an independent subsequence can be contiguous (i.e., next to each other).

Given an independent subsequence of A, we can form the sum of the numbers in this subsequence, The “maximum independent subsequence” problem is to find the maximum possible sum over all possible independent subsequences from the given array A.